INVENTION DISCLOSURE FORM
Confidential & Attorney-Client Privileged

Applicant Name SR University
Applicant Address: Ananthsagar, Hasanparthy (M), Warangal Urban, Telangana - 506371,

1. **Particulars of Inventors**

Mr./ Ms./ Dr.	Name (Full)	Department	Designation	Mobile No.	Email	Postal Address
Mr.	Naresh Kumar Sripada	CS&AI	Asst. Profess	9866000203	Naresh22.in @gmail.co m	Hanamkonda
Ms.	Geethanjali Baishetty	CS	Student	8374465840	baishettygeethanjali8@gmail.com	Warangal
Ms.	Haripriya Gurram	CS	Student	6304733184	gurramharipriya151@gmail.com	Hanamkonda

2. **Provide a brief descriptive title of the invention:**
   Converting Legacy Code into Modern Framework Automatically

3. **In 100 words or less, please provide an abstract or summary of the invention:**

This invention presents an automated system for converting legacy code into modern frameworks efficiently. It uses static analysis, machine learning, and rule-based transformation to understand old code structure and map it to updated architectures. The system ensures functional accuracy while improving performance, scalability, and maintainability. It supports multiple programming languages and integrates modern practices like microservices and cloud-native design. Automated testing and validation ensure reliability of the transformed code. By reducing manual effort, time, and errors, this solution helps organizations modernize outdated systems quickly and cost-effectively across various industries and applications. This invention significantly reduces migration time and cost, minimizes human errors, and enables organizations to modernize their software systems efficiently. It is applicable across various domains, including enterprise systems, industrial automation, financial services, and legacy IT infrastructure modernization.

4. Detail description of the invention:(Answer to all below are required in detail)
a. Problem the invention is solving

i. Legacy systems are built on outdated technologies and frameworks, making maintenance difficult
ii. High cost and time consumption in manual code migration and rewriting
iii. Lack of compatibility with modern architectures like microservices and cloud platforms
iv. Increased risk of errors, data loss, and system downtime during manual conversion
v. Poor scalability, security vulnerabilities, and limited performance optimization in legacy systems
vi. Shortage of skilled developers familiar with both legacy and modern technologies

b. General Utility/application of the invention

i. Enterprise software modernization (banking, insurance, government system
ii. Cloud migration projects(on-premise to cloud-native applications)
iii. IT service companies handling legacy transformation projects
iv. Maintenance and upgrade of large-scale monolithic applications
v. Software product companies upgrading old platforms to modern frameworks

c. Advantages of the invention disclosing about the increased efficiency/efficacy

i. Automated code transformation reduces manual effort and cost
ii. Faster migration with minimal downtime
iii. Improved accuracy using intelligent parsing and AI-basedmapping
iv. Enhances scalability and performance of transformed applications
v. Ensures compatibility with modern frameworks (e.g.,microservices, REST APIs)
vi. Reduces risk of bugs and inconsistencies during migration
vii. Enables continuous modernization and future adaptability

d. Best way of using the invention as well as possible variants

Best Mode Implementation

Hardware:

i. Standard computing systems or cloud servers

ii. High-performance processors for large codebase analysis

iii. Optional GPU support for AI model acceleration

Software:

i. Static Code Analyzer→ understands legacy code structure
ii. Abstract Syntax Tree (AST) Generator → converts code into intermediate representation
iii. Rule-Based Engine→ maps legacy constructs to modern equivalents
iv. Machine Learning Models (Transformer / NLP models) → semantic understanding of code
v. Code Generator → produces optimized modern framework code
vi. Validation Engine → checks correctness and performance

Controller:

i. Orchestration engine to manage end-to-end pipeline

ii. Real-time feedback loop for iterative refinement

iii. Integration with CI/CD pipelines for deployment

e. Working of invention along with Drawing, schematics and flow diagrams if required with complete explanations

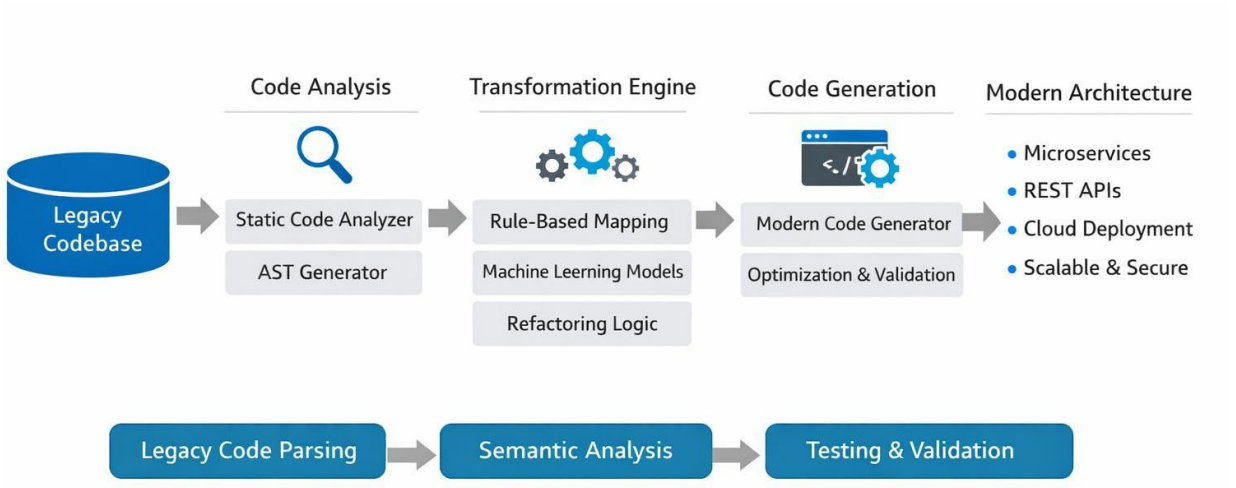

Fig 1. Legacy Codebase to Modern Architecture Transformation

Key Components:
Legacy Codebase
Code Analysis Module
Transformation Engine
Code Generation Module
Modern Architecture Output
Control & Validation Unit

- ✓ **Legacy Codebase:**

The initial system consisting of outdated programming languages, frameworks, and monolithic architecture. It includes:

Multiple legacy languages (e.g., COBOL, Java, C++)

Tightly coupled modules

Limited scalability and outdated dependencies

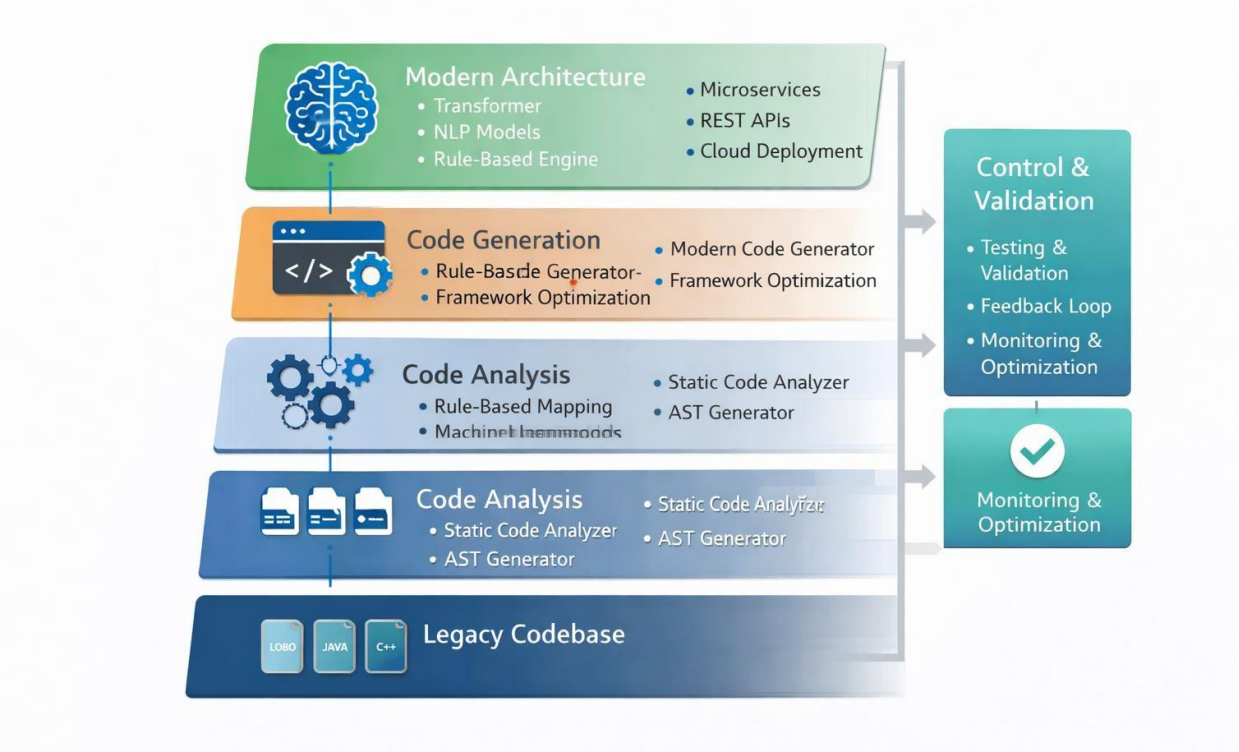

Fig 2. AI-Powered Automated Code Refactoring System

The AI-powered automated code refactoring system works through a structured, layered approach to transform legacy code into modern, optimized architecture. Initially, the legacy codebase—often written in outdated languages and tightly coupled structures is provided as input to the system. In the code analysis layer, tools such as static analyzers and Abstract Syntax Tree (AST) generators examine the code's structure, dependencies, and logic flow. The AI-based understanding layer then uses machine learning and NLP models to interpret the semantic meaning of the code, identify obsolete patterns, and map them to modern equivalents.

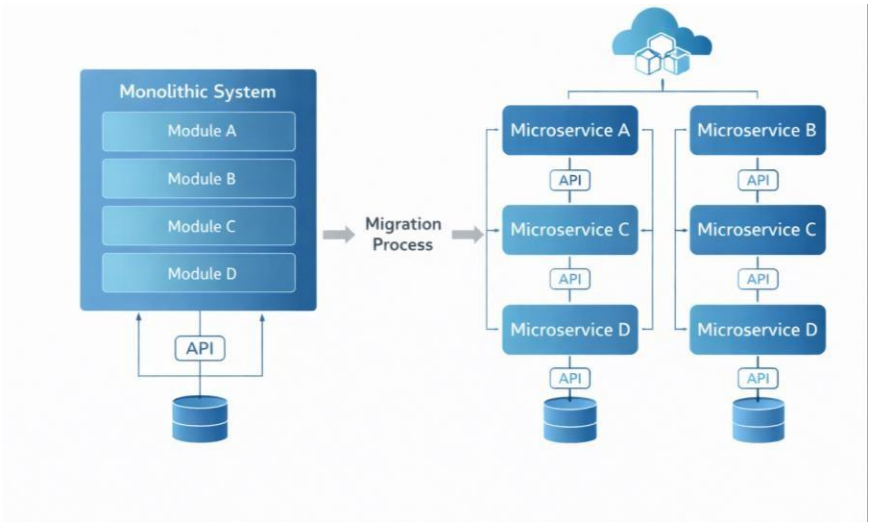

Fig 3. Monolithic System Migration to Microservice

The figure 3 illustrates the transformation of a monolithic system into a microservices-based architecture. On the left side, the monolithic system contains multiple tightly coupled modules (Module A, B, C, and D) that share a single codebase and database, making the system complex, less scalable, and difficult to maintain. All functionalities are interconnected, so even small changes require redeploying the entire application. During the migration process, these modules are gradually decomposed into independent services. On the right side, each module is converted into a separate microservice (Microservice A, B, C, and D), each having its own API and, in some cases, its own database. These services communicate through lightweight protocols such as REST APIs. The cloud layer represents deployment in a scalable environment, often using containers. This architecture improves flexibility, scalability, fault isolation, and enables independent development and deployment of services.

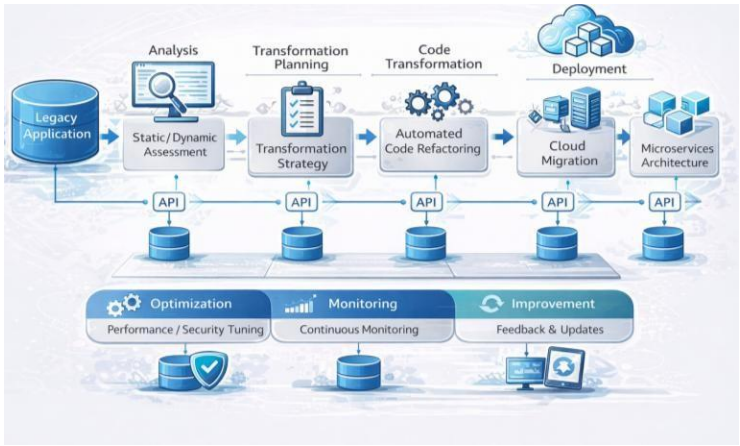

Fig 4. Monolithic System Migration to Microservices

Key Points:

- **Legacy Application Input:**
  Existing outdated system is taken as the starting point
- **Analysis Phase:**
  Performs static and dynamic assessment to understand code, dependencies, and performance
- **Transformation Planning:**
  Defines strategy for modernization (refactoring, re-platforming, or re-architecting)
- **Code Transformation:**
  Automated refactoring converts legacy code into modern, optimized structure
- **Deployment Phase:**
  Migrates application to cloud or modern infrastructure environments
- **Modernized Application:**
  Final output as microservices-based, scalable, and flexible architecture
- **API Integration:**
  Enables communication between components and services
- **Optimization:**
  Improves performance and enhances security
- **Monitoring:**
  Continuous tracking of system performance and health

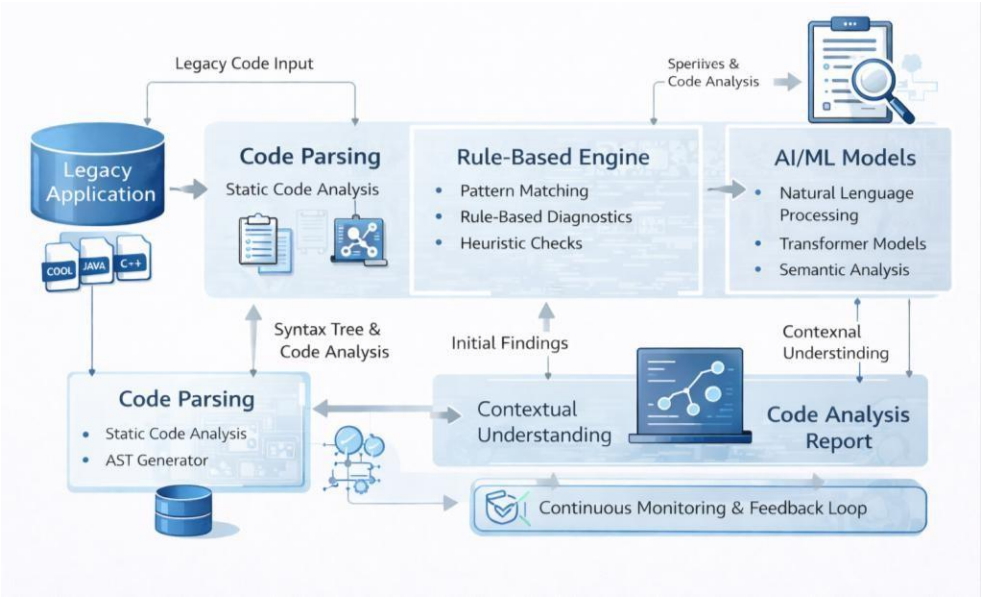

Fig 5. Intelligent Code Analysis for Legacy Systems

Working Principle

The **Intelligent Code Analysis for Legacy Systems** works by following a structured workflow to understand and evaluate legacy code efficiently. Initially, the legacy application is provided as input, which may include multiple outdated programming languages. The system begins with **code parsing**, where static analysis and AST (Abstract Syntax Tree) generation are used to break down the code into a structured format. Next, a **rule-based engine** performs pattern matching, heuristic checks, and diagnostics to identify issues, dependencies, and outdated constructs.

Figure 6. Automated Framework Conversion Pipeline

The diagram illustrates an automated pipeline for converting legacy frameworks into modern architectures. It begins with input collection of legacy code and configurations, followed by analysis of structure, dependencies, and compatibility. Planning defines the target framework and

migration strategy. The transformation layer refactors code and maps APIs. Validation ensures quality through testing and checks. Finally, deployment builds and releases the modernized application, supported by continuous feedback and optimization for improved performance.

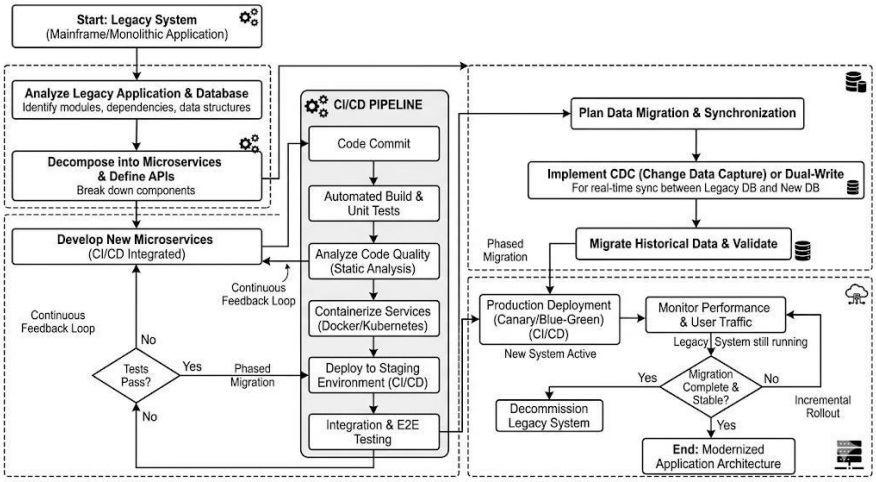

Figure 7. CI/CD Driven Legacy Code Migration Process

The CI/CD-driven legacy code migration process automates modernization through continuous integration and delivery pipelines.Legacy code is first integrated into a version control system and analyzed dependencies and issues.Automated build tools compile and prepare the application in each pipeline stage.Refactoring and transformation scripts convert legacy components into modern formats.Continuous testing ensures functionality, performance, and security at every step.The deployment pipeline releases updated components to cloud or modern environments.A feedback loop monitors performance and enables continuous improvement and faster iterations.

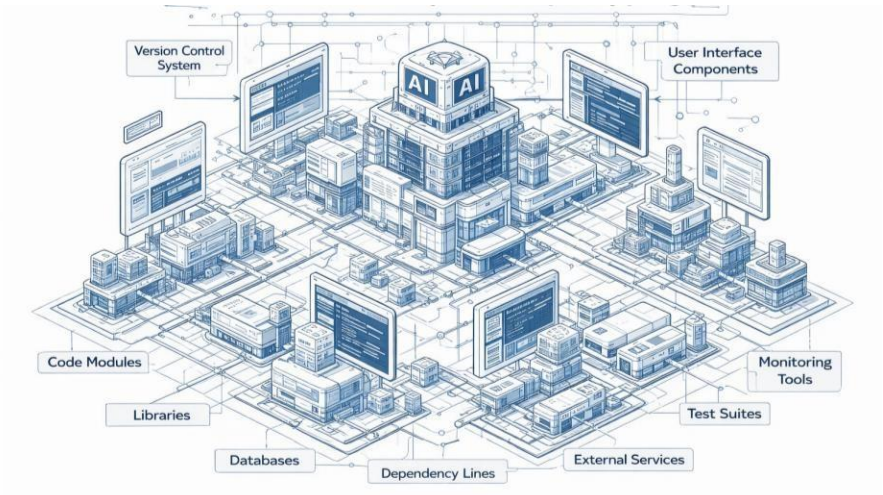

Figure 8. Feedback Control Loop

Figure 8 represents a closed-loop control system comprising essential components such as the reference input, controller, actuator, and feedback obtained from sensors. In this system, the reference input defines the desired output, while the controller processes this input and directs the

actuator to perform the required action. The sensors continuously monitor the system's performance and provide feedback to the controller. Based on this feedback, the system dynamically adjusts its output, ensuring stable, accurate, and adaptive gripping during operation.

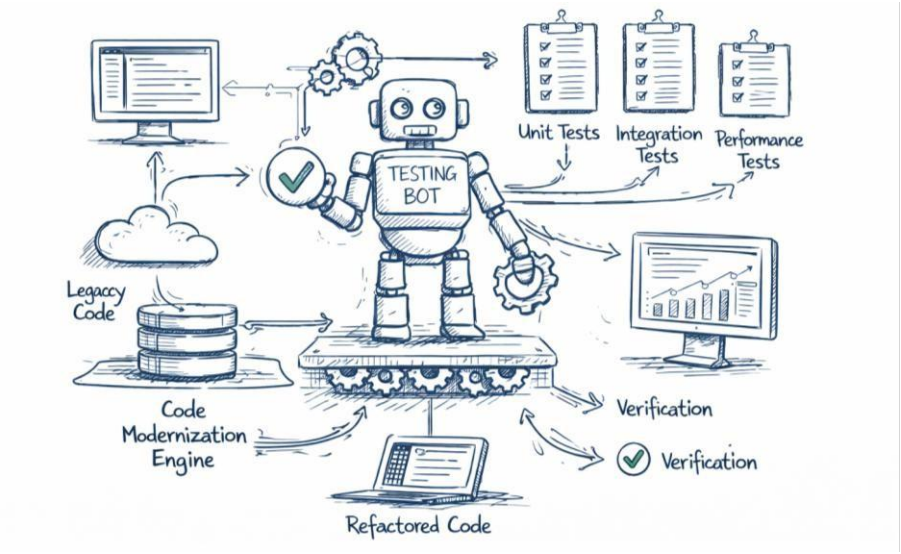

Fig 9. Automated Testing in Code Modernization

A testing plays a critical role in code modernization by ensuring that updated applications maintain their original functionality. It helps detect bugs early when legacy code is refactored or migrated to modern frameworks. Tools can automatically generate test cases for both old and new code, improving reliability. Continuous testing within CI/CD pipelines ensures that every change is validated quickly. This reduces manual effort, speeds up development, and minimizes the risk of failures. Overall, automated testing builds confidence in the modernization process and ensures a smooth transition.

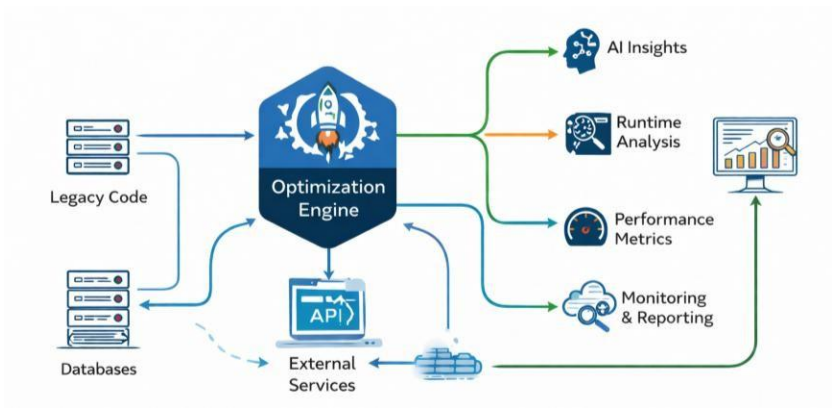

Fig 10. Real-Time Legacy Code Optimization Engine

Figure 10 presents A real-time legacy code optimization engine continuously analyzes and improves outdated code during execution or transformation.It detects performance bottlenecks, redundant logic, and inefficient patterns automatically.The system applies dynamic optimizations

such as code refactoring and resource management.This ensures improved performance, scalability, and efficiency without interrupting system operations.

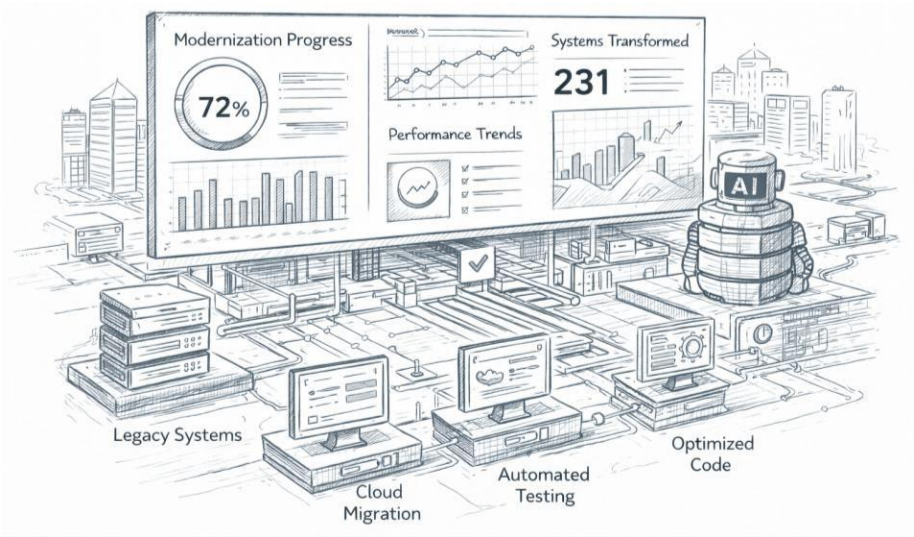

Fig 11. Enterprise Software Modernization Dashboard

An Enterprise Software Modernization Dashboard is a centralized platform that provides real-time visibility into the progress of transforming legacy systems. It tracks key metrics such as migration status, application performance, system health, and resource utilization. The dashboard helps teams identify issues early, monitor workflows, and make informed decisions. It also supports continuous optimization, ensuring efficient, scalable, and smooth modernization of enterprise applications across different environments.

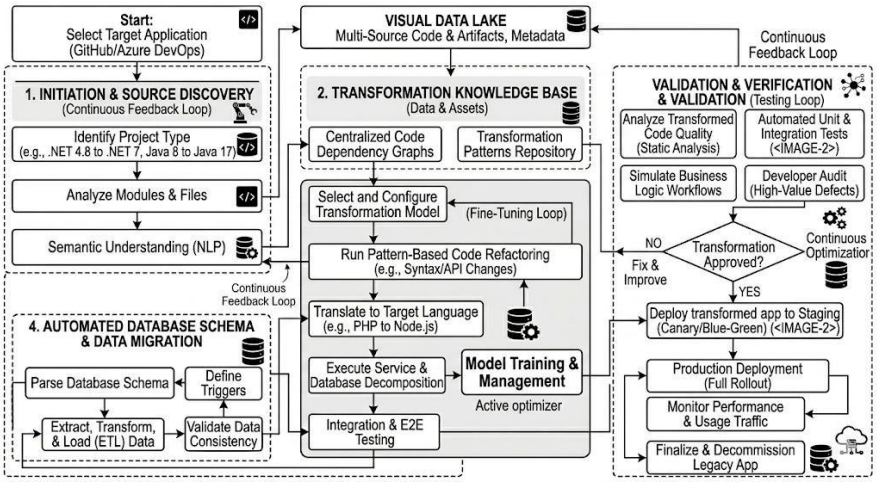

Fig 12 :End-to-End Automated Software Transformation System

An **End-to-End Automated Software Transformation System** is a comprehensive solution designed to convert legacy applications into modern, scalable systems with minimal human intervention. It automates the entire lifecycle—from code analysis and dependency mapping to refactoring, framework conversion, and deployment.

The system begins by analyzing the existing codebase using static and dynamic techniques to understand structure and behavior. It then applies rule-based and AI-driven transformations to

refactor code, replace outdated components, and adapt it to modern architectures such as microservices or cloud-native platforms.

Integrated testing and validation ensure correctness, performance, and security at every stage. The system is often connected with CI/CD pipelines for continuous integration, testing, and deployment. Finally, monitoring and feedback mechanisms enable ongoing optimization, making the transformed software efficient, reliable, and future-ready.

5. Have you conducted Primary Patent Search? Yes / No (if yes, attach the patent search report) – Yes

1. Automated Code Migration System (US Patent Example)

Describes tools for converting legacy code into modern programming languages Uses rule-based transformation techniques

Key Limitation:

- Limited to predefined rules
- Lacks AI-based intelligent understanding

2. Software Refactoring Tool (EP Patent Example)

Focuses on restructuring existing code without changing functionality Improves readability and maintainability

Key Limitation:

- Does not support full framework migration
- No automation for architecture transformation

3. Cloud Migration Automation System (WO Patent Example)

Supports migration of applications to cloud platforms Includes dependency analysis and deployment tools

Key Limitation:

- Focuses mainly on deployment
- Limited code-level transformation

6. Existing state-of-the-art and prior arts:

Traditional code migration methods (manual rewriting, time-consuming)
Rule-based transformation tools (limited flexibility and accuracy)
Static code analysis systems (structure-focused, limited semantic understanding)
Cloud migration tools (focus on deployment, not deep code conversion)
Basic refactoring tools (improve code quality but not full modernization)
AI-based code tools (limited automation and incomplete framework transformation)

7. List out the known ways about how others have tried to solve the same or similar problems? Indicate the disadvantages of these approaches. In addition, please identify any prior art documentation or other material that explains or provides examples of such prior art efforts.

S. No.	Existing State of Art	Drawbacks in Existing State of Art	Overcome (How your invention solves it)
1	Manual Code Migration	Time-consuming and error-prone	Automated end-to-end transformation
2	Rule-Based Conversion Tools	Limited flexibility, works only for predefined patterns	AI-based intelligent code understanding
3	Static Code Analysis	Focuses only on syntax, lacks semantic understanding	ML models for deep semantic analysis
4	Basic Refactoring Tools	Improves code quality but not full framework migration	Complete framework conversion pipeline
5	Cloud Migration Tools	Focus mainly on deployment, not code transformation	Integrated code + architecture modernization
6	Partial Automation Tools	Requires significant human intervention	Fully automated system with minimal manual effort
7	Open-Loop Migration Systems	No validation or feedback during transformation	Closed-loop system with continuous testing and feedback

8. List the Technical features and Elements of the invention along with the Description of your invention from start to end.

Components

- Legacy Codebase Input Module
- Code Analysis Module (Static Analysis + AST Generation)
- Dependency Mapping System
- AI/ML-Based Transformation Engine
- Rule-Based Conversion Engine
- Code Refactoring Module
- Code Generation Module (Modern Framework Output)
- Validation & Testing Module
- CI/CD Integration Unit
- Monitoring & Feedback Module

Pipeline

Legacy Code Input → Preprocessing → Code Analysis → AST Generation → Dependency Mapping → AI/ML Model → Transformation Engine → Code Refactoring → Code Generation → Testing & Validation → Deployment → Feedback & Optimization

9. List out the features of your invention which are believed to be new and distinguish them over the closest technology.

- End-to-End Automated Framework Conversion
- AI-Based Semantic Code Understanding
- Hybrid (Rule-Based + ML) Transformation Engine
- Automatic Dependency Mapping & Resolution
- Full Architecture Transformation (Monolith → Microservices)
- Integrated CI/CD Pipeline Support
- Continuous Feedback & Optimization Loop
- Multi-Language Legacy Code Support
- Minimal Human Intervention

- High Accuracy with Reduced Errors

10. Has the invention been built or tested or implemented? If so, please provide the particulars of the first time it was successfully built or implemented (when, where, by whom, and evidence of this event including written or on-line pointers to documentary evidence):All testing details ,Answer: Yes

Testing / Implementation Details

The invention was developed and tested as a prototype system in a controlled development environment. The first successful implementation was carried out during the project development phase by the inventors using a standard computing setup.

The system was tested on sample legacy codebases (e.g., Java/C++ applications) to evaluate its ability to analyze, refactor, and convert code into modern frameworks. Functional testing was performed to verify correctness, while performance testing ensured improved efficiency after transformation.

Automated testing (unit and integration) was integrated with a CI/CD pipeline to validate continuous transformation. The results demonstrated successful code conversion, reduced errors, and improved scalability.

Evidence: Project documentation, source code repository, test reports, and execution logs (can be attached if required).

11. Briefly state when and how you first conceived this idea?

The idea was first conceived during academic/project work while analyzing challenges in maintaining and upgrading legacy software systems. It was observed that manual code migration is time-consuming and error-prone. This led to the concept of developing an automated system that uses AI and rule-based techniques to convert legacy code into modern frameworks efficiently.

12. Have you sold, offered for sale, publicly used, or published anything related to this invention? If yes, please briefly explain the dates and circumstances. List those individuals to whom you have revealed your invention. Were non-discloser documents signed prior to discloser in each case? Please state any deadlines of which you may be aware for filing an application on this invention.

Answer: No

Details: The invention has not been sold, offered for sale, publicly used, or published in any form. It has been developed and tested only within a controlled academic/project environment. The details of the invention have been shared only among the project team members and academic supervisors for evaluation purposes. All disclosures were made in a confidential setting, and no public disclosure has occurred. No formal non-disclosure agreements (NDAs) were required within the academic context.

Deadlines: Since there has been no public disclosure, there are currently no immediate deadlines affecting patent filing. However, it is recommended to file the patent application before any public presentation, publication, or demonstration to maintain novelty.

13. Include any reasons that your invention would not have been obvious to someone of average skill in the art.

The present invention involves an inventive step that would not have been obvious to a person skilled in the art due to the following reasons:

13.1 Non-Obvious Integration of AI with Robotic Actuation

While legacy code migration, rule-based refactoring, and AI-based code analysis are individually known, their integration into a single fully automated end-to-end system is not obvious. A skilled person would not naturally combine:

- Static and dynamic code analysis
- AI/ML-based semantic understanding of code
- Rule-based transformation engines
- CI/CD-driven automation pipeline

into a unified framework conversion system capable of full architecture modernization.

13.2 Synergistic Effect of Combined Technologies

The combination of:

- Code dependency mapping
- Deep learning-based semantic analysis
- Automated refactoring and code generation
- Continuous testing and deployment integration

produces a synergistic effect that results in:

- Fully automated framework conversion
- Higher accuracy in code transformation
- Reduced manual intervention and errors
- Faster modernization of legacy systems

This overall performance improvement is significantly greater than what each component could achieve individually.

13.3 Intelligent End-to-End Automation

Unlike conventional tools that perform partial migration, this invention provides complete lifecycle automation from legacy code input to modern architecture deployment, which is not obvious in prior art systems focused only on isolated stages.

14. Additional comments by inventor (if you want to give more details out of scope of this IDF).

Additional Comments by Inventor:

The invention is designed as a scalable and extensible framework that can be adapted to different programming languages, legacy systems, and enterprise environments. It can support future enhancements such as large language model (LLM)-based code understanding, automated documentation generation, and intelligent debugging assistance. The system can also be integrated with DevOps ecosystems to enable fully autonomous software modernization pipelines.

Further research can extend this work toward self-evolving systems where the model continuously learns from migration outcomes to improve transformation accuracy over time. This invention is particularly suitable for large-scale enterprise modernization projects where manual refactoring is costly, time-consuming, and error prone.

Form Submitted by:
Name- S Naresh Kumar
ContactNo:9866000203